

Reinforcement Learning

Class 5

Model – Free Control:

MC and TD; online and offline learning; Q-learning

Mostly based on D. Silver (UCL) lecture 5 from 2015
and sections 5.3-5.5, 5.7/6.4-6.6/7.2/12.7 from Sutton & Barto's book

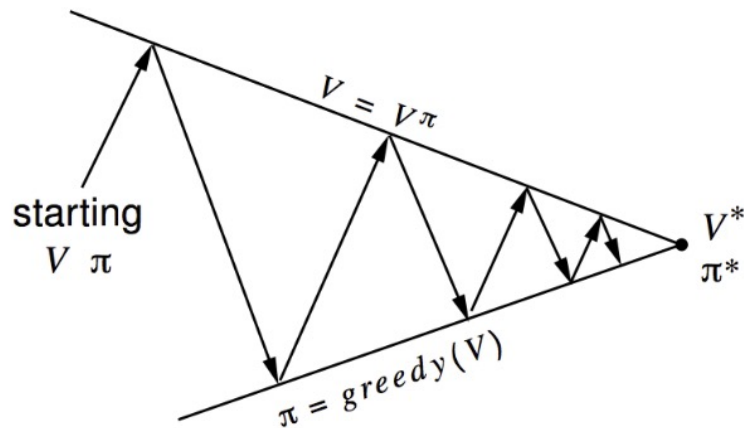
Context

- Last classes – we presented MDPs as the mathematical framework underlying RL problems, next Dynamic Programming as a way to solve known MDPs
- We then moved to methods to **estimate the value functions of unknown MDPs (model-free prediction)**: Monte-Carlo and temporal-difference learning
- This class: we will study methods to optimise the value functions of unknown MDPs (**model-free control**), moving from policy evaluation to policy improvement (optimize policies)
- So, after this class, you will be able to start **developing agents that can handle unknown environments and optimize policies**
- Next classes, we will learn how to scale up and improve for more challenging environments, but this lecture defines the core algorithms for control

Applications

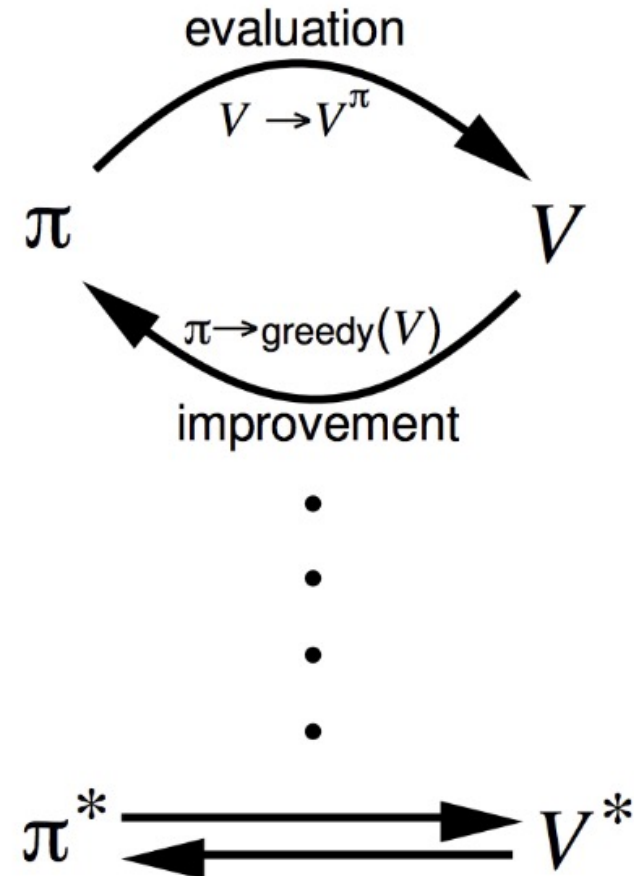
- Example problems that can be modelled as MDPs
 - Elevator
 - Bioreactor
 - Helicopter
 - Aeroplane Logistics
 - Robocup Soccer
 - Quake
 - Portfolio management
 - Protein Folding
 - Robot walking
 - Game of Go ... and many more !
- For most of these problems, either:
 - MDP model is **unknown**, but experience can be **sampled**
 - MDP model is **known**, but is **too big to use**, except by samples
- **Model-free control** can address these problems

Generalized Policy iteration (from class 3)

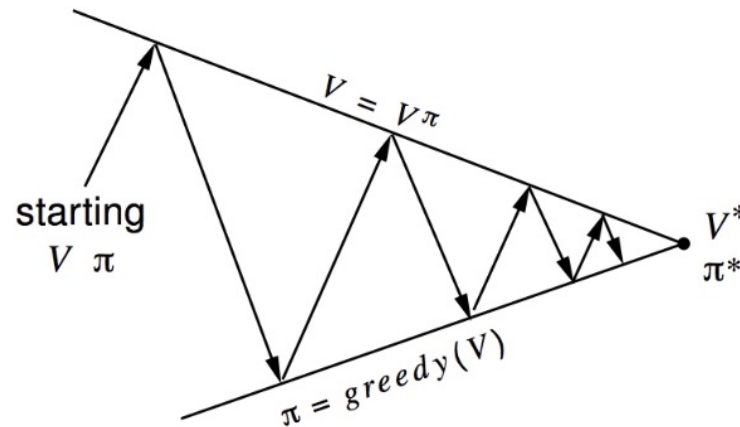


$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

- E- Policy **evaluation** – estimate v_{π} –
any policy evaluation algorithm (e.g. iterative)
- I - Policy **improvement** – generate $\pi' \geq \pi$ -
any policy improvement algorithm (e.g. greedy)



Generalized policy iteration with MC evaluation



Policy **evaluation** – estimate v_π –
Monte Carlo policy evaluation (last class)- v_π

Policy **improvement** – generate $\pi' \geq \pi$ -
Greedy policy improvement ?

Is this possible ?

Model-Free Policy Iteration Using Action-Value Function

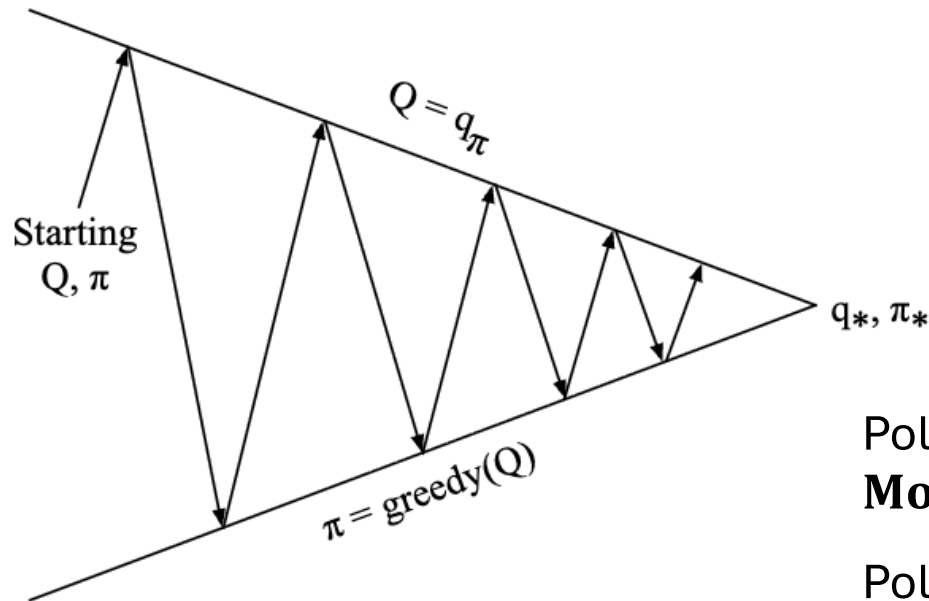
- Greedy policy improvement over $V(s)$ requires model of MDP

$$\pi'(s) = \operatorname{argmax}_{a \in \mathcal{A}} \mathcal{R}_s^a + \mathcal{P}_{ss'}^a V(s')$$

- Alternative: **Greedy policy improvement over $Q(s,a)$ is model-free**

$$\pi'(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a)$$

Generalised Policy Iteration with Action-Value Function



Policy **evaluation** – $Q = q_\pi$ –
Monte Carlo policy evaluation

Policy **improvement** – generate $\pi' \geq \pi$ –
Greedy policy improvement ?

Does it work ?

Example of Greedy Action Selection

There are two doors in front of you.

- You open the left door and get reward 0 - $V(\text{left}) = 0$
 - You open the right door and get reward +1 - $V(\text{right}) = +1$
 - You open the right door and get reward +3 - $V(\text{right}) = +2$ ($MC - \text{mean}$)
 - You open the right door and get reward +2 - $V(\text{right}) = +2$
-
- Are you sure you've chosen the best door?
 - We are again facing **exploration vs exploitation**

Example of a bandit problem !

We know we should explore the non-greedy option

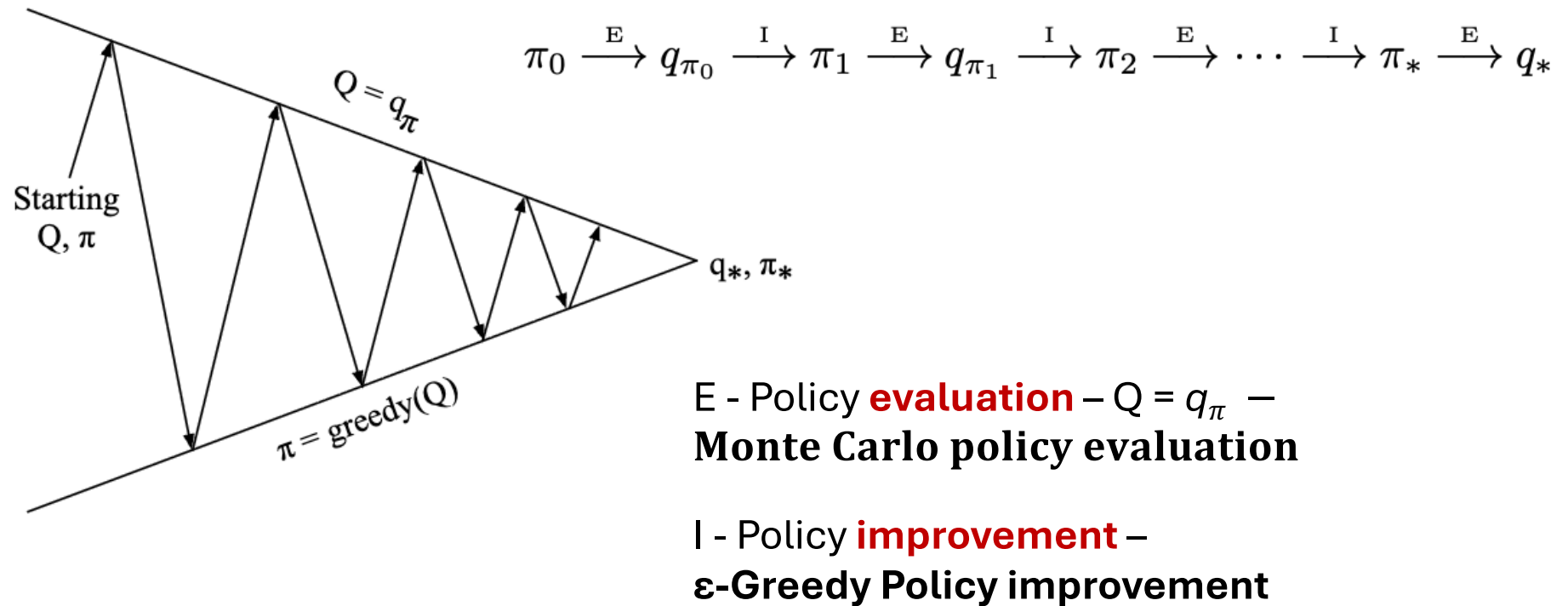


ϵ -Greedy Exploration

- Simplest idea for ensuring continual exploration, already seen in our first class (K-bandits)
- All m actions are tried with non-zero probability
- With probability $1-\epsilon$ choose the greedy action
- With probability ϵ choose an action at random

$$\pi(a|s) = \begin{cases} \epsilon/m + 1 - \epsilon & \text{if } a^* = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a) \\ \epsilon/m & \text{otherwise} \end{cases}$$

Monte Carlo Policy Iteration



ϵ -Greedy Policy improvement

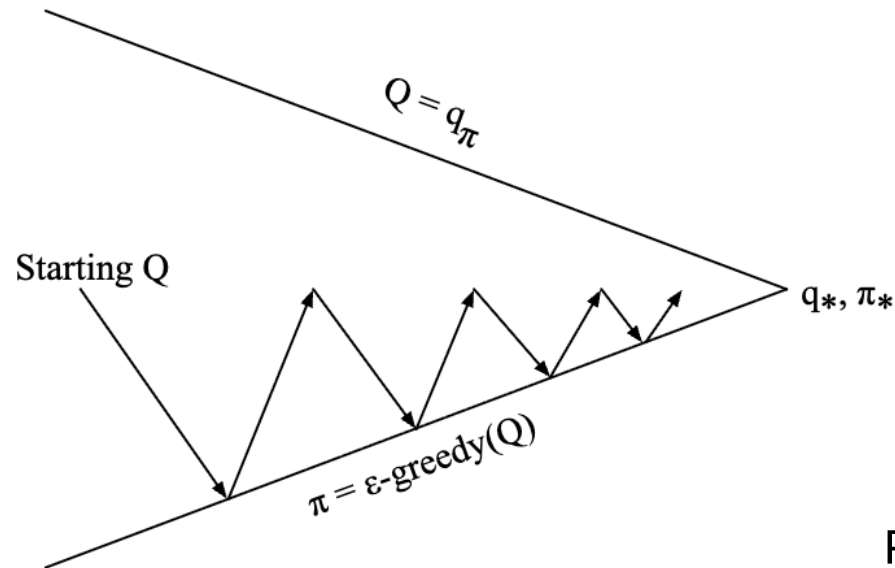
- **Theorem:** for any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to q_π is an improvement $v_{\pi'}(s) \geq v_\pi(s)$

$$\begin{aligned} q_\pi(s, \pi'(s)) &= \sum_{a \in \mathcal{A}} \pi'(a|s) q_\pi(s, a) \\ &= \epsilon/m \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} q_\pi(s, a) \\ &\geq \epsilon/m \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \epsilon/m}{1 - \epsilon} q_\pi(s, a) \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a) = v_\pi(s) \end{aligned}$$

Assumes an
infinite
number of
episodes in
policy
evaluation

Therefore from policy improvement theorem: $v_{\pi'}(s) \geq v_\pi(s)$

Monte Carlo control



We can update Q
after **every single episode**

Policy **evaluation** – $Q \approx q_\pi$ –
Monte Carlo policy evaluation

Policy **improvement** –
 ϵ -Greedy Policy improvement

Greedy in the Limit with Infinite Exploration (GLIE)

- All state-action pairs are explored infinitely many times

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- The policy converges on a greedy policy (Bellman optimality)

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbf{1}(a = \operatorname{argmax}_{a' \in \mathcal{A}} Q_k(s, a'))$$

- For example, ε -greedy is GLIE if ε reduces to zero at $\varepsilon_k = 1/k$

GLIE Monte Carlo control

- Sample k-th episode using π : $\{S_1, A_1, R_2, \dots, S_T\} \sim \pi$
- For each state S_t and action A_t in the episode

$$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} (G_t - Q(S_t, A_t))$$

- Improve policy based on new action-value function (k episodes)

$$\epsilon \leftarrow 1/k$$

$$\pi \leftarrow \epsilon\text{-greedy}(Q)$$

Theorem: GLIE MC control converges to the optimal action-value:

$$Q(s, a) \rightarrow q_*(s, a)$$

MC control - algorithm

On-policy first-visit MC control (for ε -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

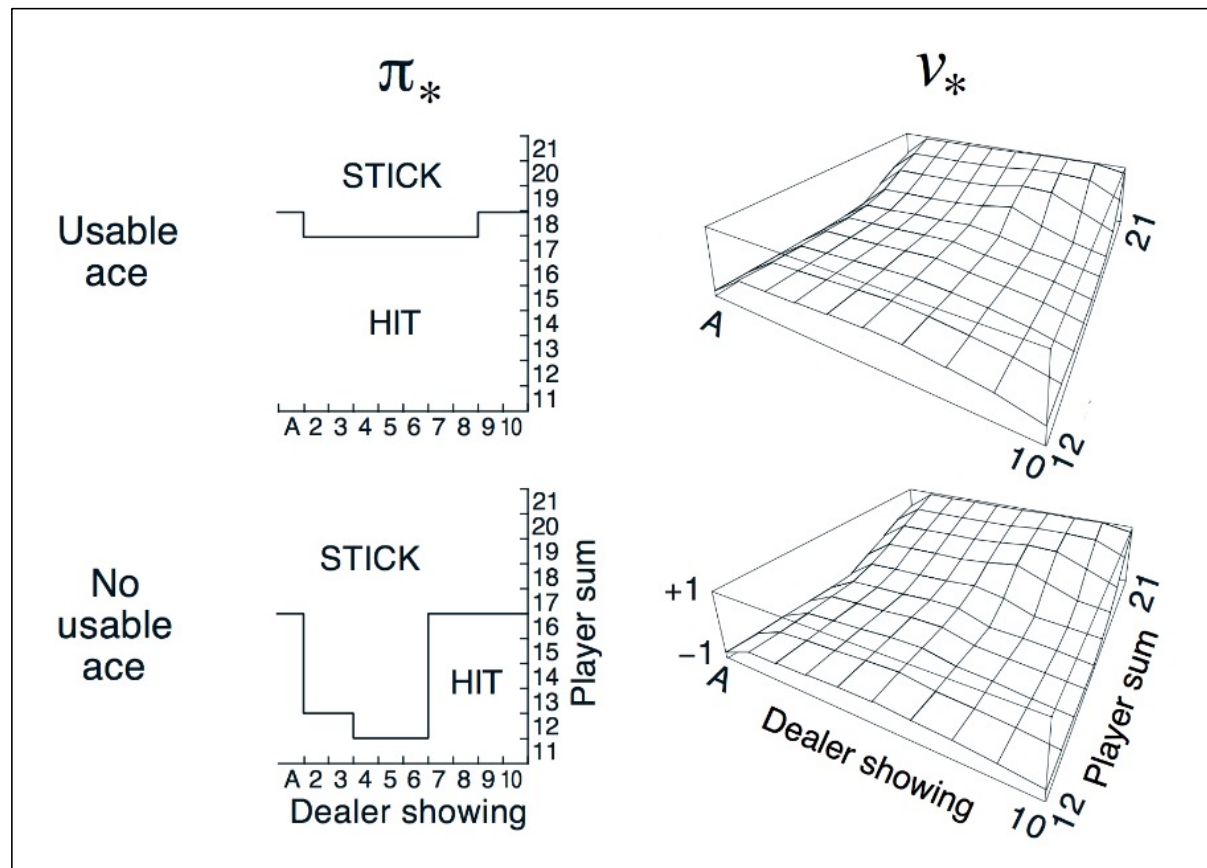
ε -soft policy

Any policy where, for some $\varepsilon > 0$, for all states and actions:

$$\pi(a|s) \geq \frac{\varepsilon}{|\mathcal{A}(s)|}$$

Pseudo-code

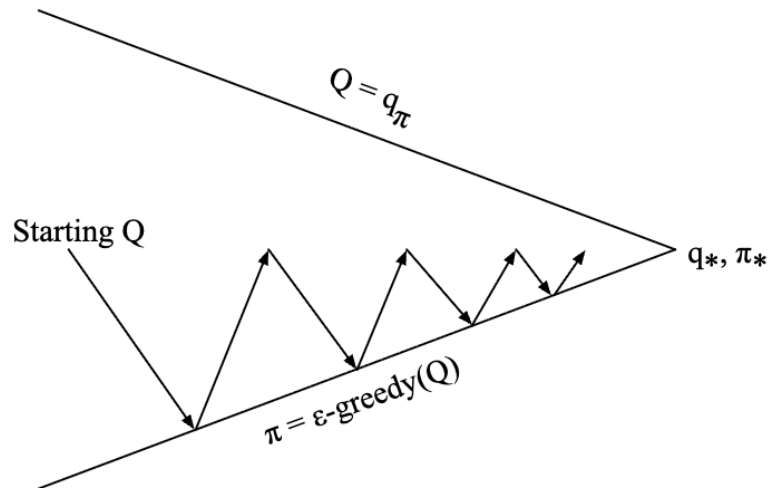
MC control in the Blackjack



Monte Carlo vs Temporal Difference - Control

- **Temporal-difference (TD) learning** has several advantages over Monte-Carlo (MC)
 - Lower variance
 - Online (continuous)
 - Incomplete sequences
- Natural idea: use **TD instead of MC in our control loop**
 - **Apply TD to $Q(S,A)$**
 - **Use ϵ -greedy policy improvement**
 - Update every time-step

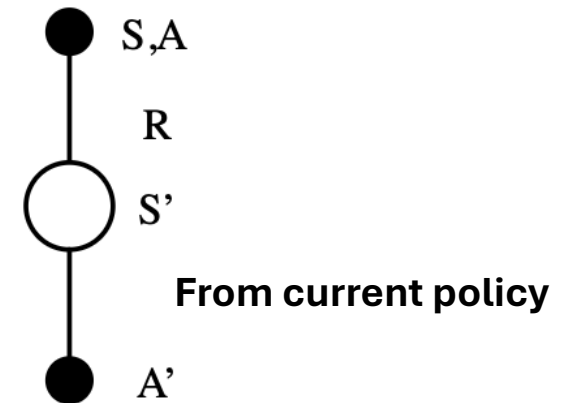
Sarsa – updating action-value functions



Policy **evaluation** – $Q \approx q_\pi$ – Sarsa

Policy **improvement** –
 ϵ -Greedy Policy improvement

Sarsa updates for prediction – TD(0)



$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

updates done in all steps; if S' is terminal – $Q(S', A') = 0$

Sarsa – algorithm for control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Pseudo-code

Sarsa - convergence

Theorem

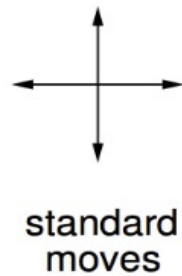
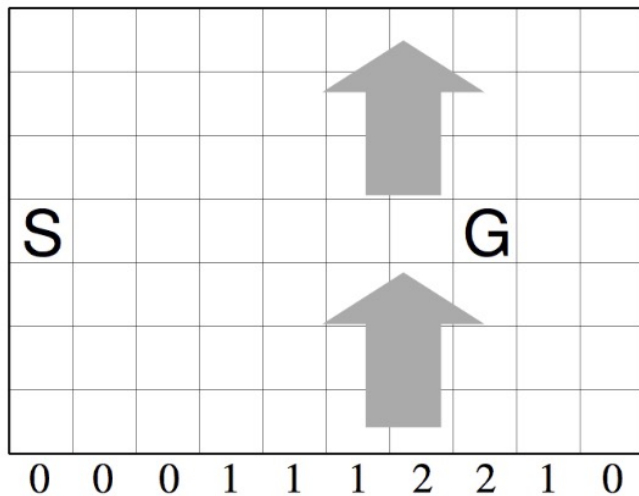
Sarsa converges to the optimal action-value function, $Q(s, a) \rightarrow q_(s, a)$, under the following conditions:*

- *GLIE sequence of policies $\pi_t(a|s)$*
- *Robbins-Monro sequence of step-sizes α_t*

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$

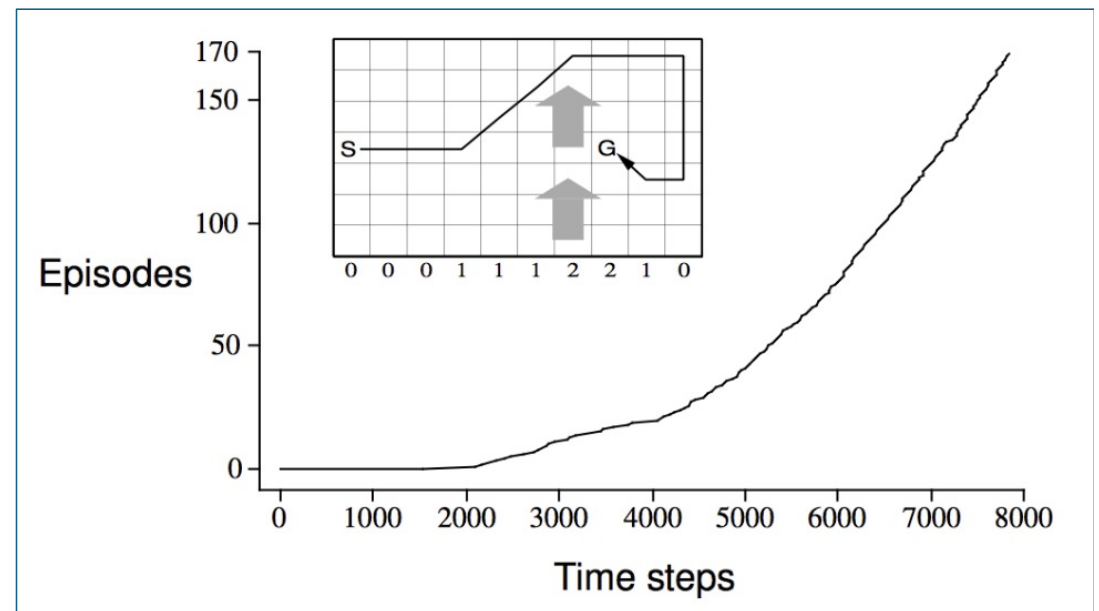
$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Example – Windy Gridworld



- Rewards = -1 per time-step until reaching goal
- Undiscounted

SARSA



n-step Sarsa

- Considering the following n-step returns for $n = 1, 2, \dots, \infty$:

$$\begin{array}{ll} n = 1 & \text{(Sarsa)} \quad q_t^{(1)} = R_{t+1} + \gamma Q(S_{t+1}) \\ n = 2 & q_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(S_{t+2}) \\ & \vdots \\ n = \infty & \text{(MC)} \quad q_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T \end{array}$$

- Define the n-step Q-return

$$q_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n})$$

- N-step SARSA updates $Q(s,a)$ towards the n-step Q-return

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^{(n)} - Q(S_t, A_t) \right)$$

n-step Sarsa

1-step Sarsa
aka Sarsa(0)



2-step Sarsa



3-step Sarsa



...

n-step Sarsa



∞ -step Sarsa
aka Monte Carlo



n-step Sarsa - algorithm

n-step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize π to be ε -greedy with respect to Q , or to a fixed given policy

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$, a positive integer n

All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

 Select and store an action $A_0 \sim \pi(\cdot | S_0)$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

 else:

 Select and store an action $A_{t+1} \sim \pi(\cdot | S_{t+1})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$

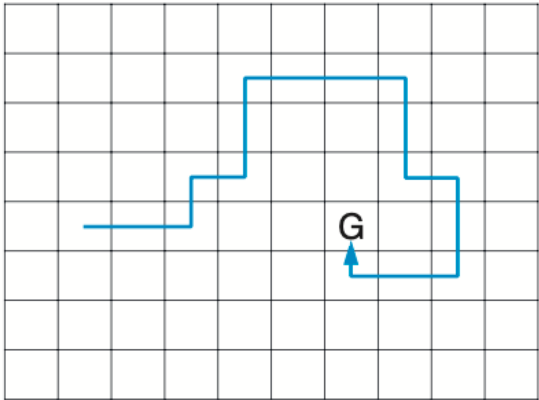
 If π is being learned, then ensure that $\pi(\cdot | S_\tau)$ is ε -greedy wrt Q

 Until $\tau = T - 1$

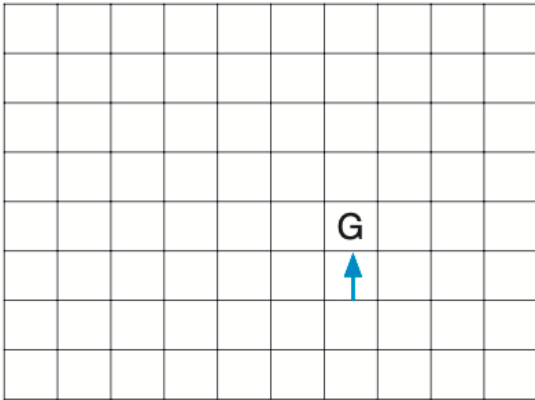
Pseudo-code

Example – Windy Gridworld – n-step Sarsa

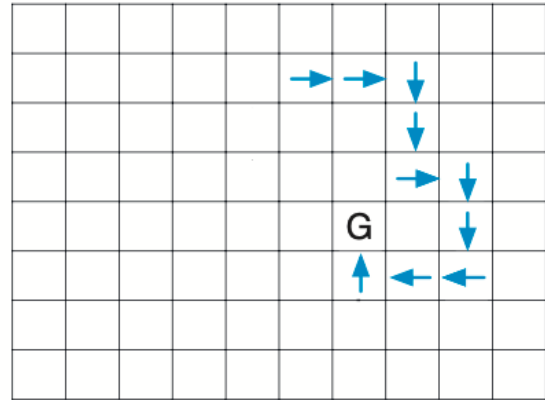
Path taken



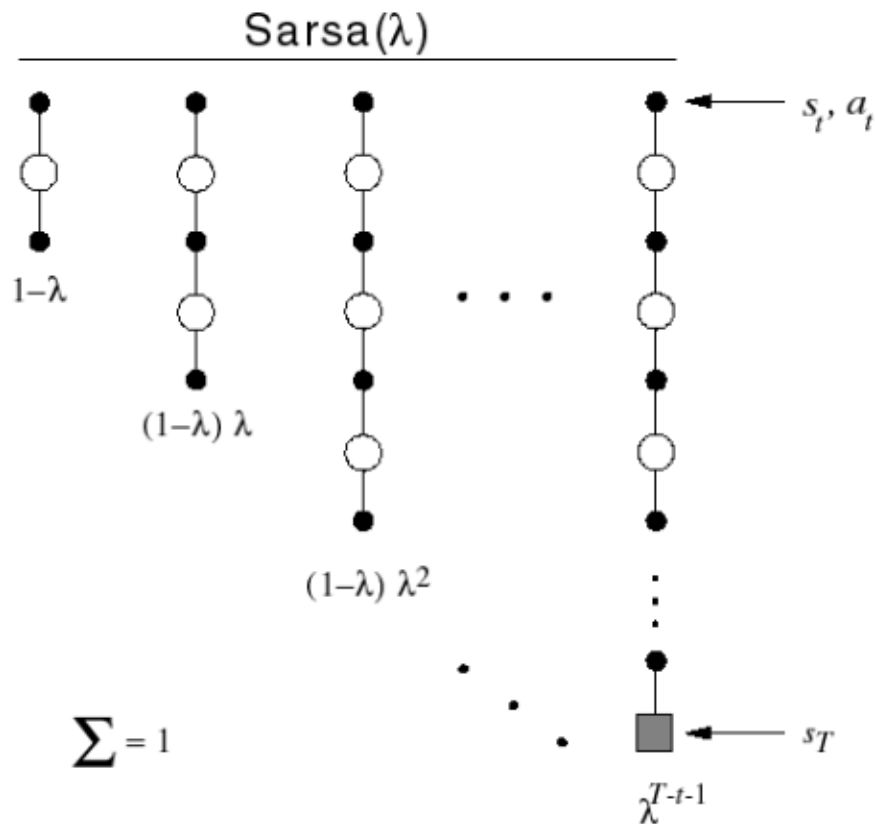
Action values increased by one-step Sarsa



Action values increased by 10-step Sarsa



Forward view Sarsa(λ)



- The q^λ return combines all n-step Q-returns $q_t^{(n)}$

- Using weight $(1-\lambda) \lambda^{n-1}$

$$q_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} q_t^{(n)}$$

- Forward-view SARSA(λ)

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^\lambda - Q(S_t, A_t) \right)$$

Backward view Sarsa(λ)

- Just like TD(λ), we use **eligibility traces** in an online algorithm
- But SARSA(λ) has one eligibility trace for each state-action pair

$$E_0(s, a) = 0$$

$$E_t(s, a) = \gamma\lambda E_{t-1}(s, a) + \mathbf{1}(S_t = s, A_t = a)$$

- $Q(s, a)$ is updated for every state s and action a in proportion to TD-error δ_t and eligibility trace $E_t(s, a)$

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

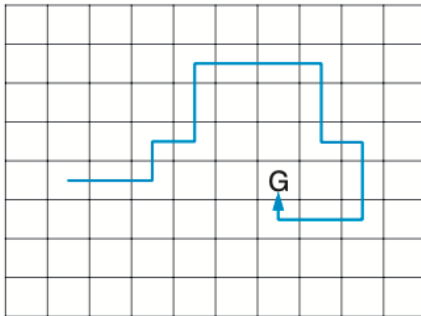
$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E_t(s, a)$$

Sarsa(λ) - algorithm

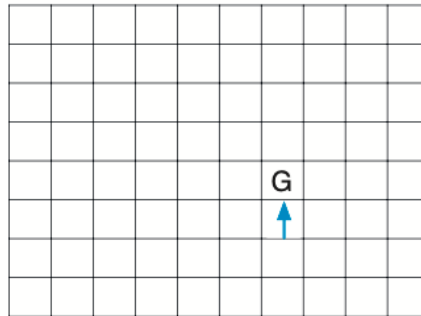
```
Initialize  $Q(s, a)$  arbitrarily, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
Repeat (for each episode):
     $E(s, a) = 0$ , for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
    Initialize  $S, A$ 
    Repeat (for each step of episode):
        Take action  $A$ , observe  $R, S'$ 
        Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
         $\delta \leftarrow R + \gamma Q(S', A') - Q(S, A)$ 
         $E(S, A) \leftarrow E(S, A) + 1$ 
        For all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ :
             $Q(s, a) \leftarrow Q(s, a) + \alpha \delta E(s, a)$ 
             $E(s, a) \leftarrow \gamma \lambda E(s, a)$ 
         $S \leftarrow S'; A \leftarrow A'$ 
    until  $S$  is terminal
```

Example – Windy Gridworld – Sarsa(λ)

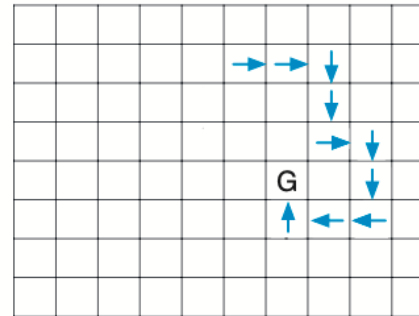
Path taken



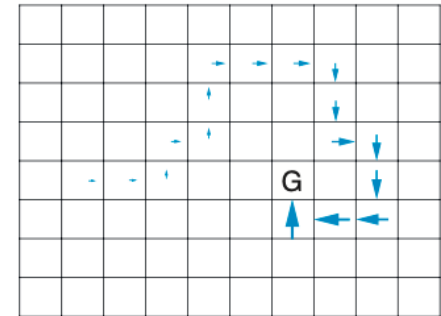
Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



Action values increased
by Sarsa(λ) with $\lambda=0.9$



On-policy and Off-policy

- **On-policy learning**

- “Learn on the job”
- Learn from policy while following it and trying to improve it
- Learn about policy π from experience sampled from π

- **Off-policy learning**

- “Look over someone’s shoulder”
- Learn about policy π from experience sampled from μ

Off-policy learning

- Evaluate **target policy** $\pi(a|s)$ to compute $v_\pi(s)$ or $q_\pi(s,a)$
- While following **behaviour policy** $\mu(a|s)$

$$\{S_1, A_1, R_2, \dots, S_T\} \sim \mu$$

- Why is this important?
 - Learn from observing humans or other agents
 - Re-use experience generated from old policies $\pi_1, \pi_2, \dots, \pi_{t-1}$
 - Learn about **optimal policy** while following an **exploratory policy** (previous cases we were always learning a near-optimal policy that still explores)
 - Learn about multiple policies while following one policy
 - Off policy methods are often **of greater variance** and are **slower** to converge, but **more powerful and general**
 - On policy is, actually, a special case of off policy where both policies are the same

Importance sampling

- For off-policy methods the behaviour policy needs to be stochastic, while the target policy may be deterministic
- Many off-policy methods use **importance sampling** – technique to estimate expected values under one distribution given samples from another
- Estimate the expectation of a different distribution

$$\begin{aligned}\mathbb{E}_{X \sim P}[f(X)] &= \sum P(X)f(X) \\ &= \sum Q(X) \frac{P(X)}{Q(X)} f(X) \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]\end{aligned}$$

Importance sampling for Off-policy Monte Carlo

- Use returns generated from μ to evaluate π
- Weight return G_t according to similarity between policies
- Multiply importance sampling corrections along whole episode

$$G_t^{\pi/\mu} = \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)} \frac{\pi(A_{t+1}|S_{t+1})}{\mu(A_{t+1}|S_{t+1})} \cdots \frac{\pi(A_T|S_T)}{\mu(A_T|S_T)} G_t$$

Importance
sampling
ratio



- Update value towards *corrected* return

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t^{\pi/\mu} - V(S_t) \right)$$

- Cannot use if **μ is zero** when **π is non-zero**
- Importance sampling can dramatically increase variance – in practice can be a very bad idea !

Off-policy MC prediction – full algorithm

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \in \mathbb{R}$ (arbitrarily)

$C(s, a) \leftarrow 0$

Loop forever (for each episode):

$b \leftarrow$ any policy with coverage of π

Generate an episode following b : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$, while $W \neq 0$:

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$

$W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

Behavioral policy
represented by b

W – represent importance
sampling ratios

C – used for a weighted
importance sampling

Method implemented
episode by episode
(incremental)

Pseudo-code

Off-policy MC control – full algorithm

Off-policy MC control, for estimating $\pi \approx \pi_*$

```
Initialize, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ :  
   $Q(s, a) \in \mathbb{R}$  (arbitrarily)  
   $C(s, a) \leftarrow 0$   
   $\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a)$  (with ties broken consistently)  
  
Loop forever (for each episode):  
   $b \leftarrow$  any soft policy  
  Generate an episode using  $b$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$   
   $G \leftarrow 0$   
   $W \leftarrow 1$   
  Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :  
     $G \leftarrow \gamma G + R_{t+1}$   
     $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$   
     $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$   
     $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$  (with ties broken consistently)  
    If  $A_t \neq \pi(S_t)$  then exit inner Loop (proceed to next episode)  
     $W \leftarrow W \frac{1}{b(A_t|S_t)}$ 
```

Aims to achieve an **optimal policy** !!

Behavioral policy represented by b

Pseudo-code

Importance sampling for Off-policy TD

- Use TD targets generated from μ to evaluate π
- Weight TD target $R + \gamma V(S')$ by importance sampling
- Only need a single importance sampling correction

$$V(S_t) \leftarrow V(S_t) + \alpha \left(\frac{\pi(A_t|S_t)}{\mu(A_t|S_t)} (R_{t+1} + \gamma V(S_{t+1})) - V(S_t) \right)$$

- Much lower variance than Monte-Carlo importance sampling
- Policies only need to be similar over a single step

Off-policy n-step Sarsa - algorithm

Off-policy n-step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize π to be greedy with respect to Q , or as a fixed given policy

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

 Select and store an action $A_0 \sim b(\cdot|S_0)$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

 else:

 Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n-1, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ ($\rho_{\tau+1:t+n-1}$)

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \rho [G - Q(S_\tau, A_\tau)]$

 If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q

 Until $\tau = T - 1$

Pseudo-code

Q-Learning

- We now consider **off-policy learning of action-values** $Q(s,a)$
- **No importance sampling is required**
- Next action is chosen using **behaviour** policy $A_{t+1} \sim \mu(\cdot|S_t)$
- But we consider alternative successor action $A' \sim \pi(\cdot|S_t)$ **for target policy**, and **update $Q(S_t, A_t)$ towards value of alternative action**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t))$$

Early method proposed by Watkins in 1989

Off-policy control with Q-learning

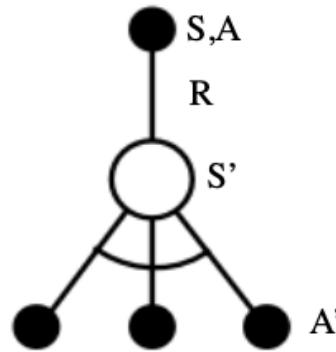
- We now allow both behaviour and target policies to improve
- The target policy π is **greedy** w.r.t. $Q(s,a)$

$$\pi(S_{t+1}) = \operatorname{argmax}_{a'} Q(S_{t+1}, a')$$

- The behaviour policy μ is e.g. **ϵ -greedy** w.r.t. $Q(s,a)$
- The Q-learning target then simplifies:

$$\begin{aligned} & R_{t+1} + \gamma Q(S_{t+1}, A') \\ &= R_{t+1} + \gamma Q(S_{t+1}, \operatorname{argmax}_{a'} Q(S_{t+1}, a')) \\ &= R_{t+1} + \max_{a'} \gamma Q(S_{t+1}, a') \end{aligned}$$

Q-Learning control algorithm (aka Sarsamax)



$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right)$$

Theorem

*Q-learning control converges to the optimal action-value function,
 $Q(s, a) \rightarrow q_*(s, a)$*

Q-Learning algorithm for off-policy control

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

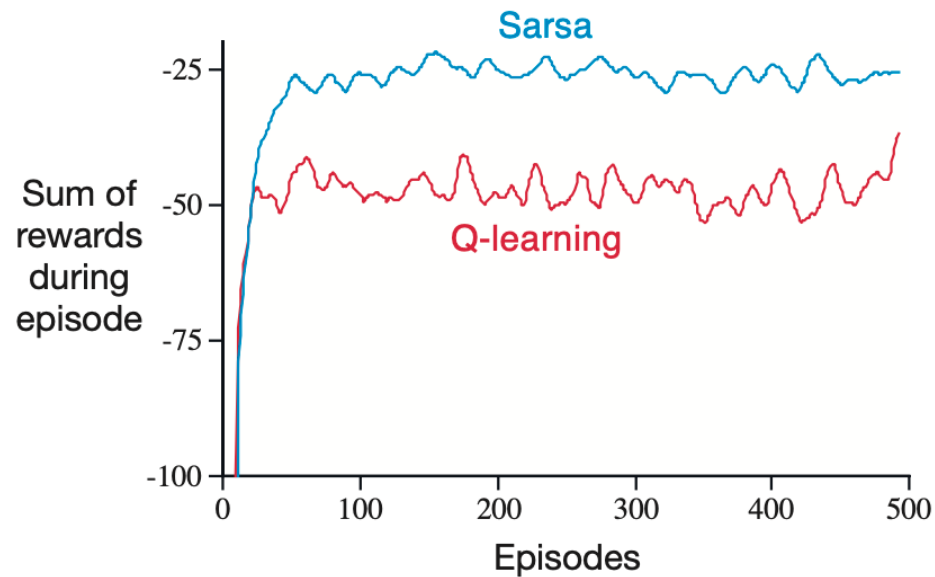
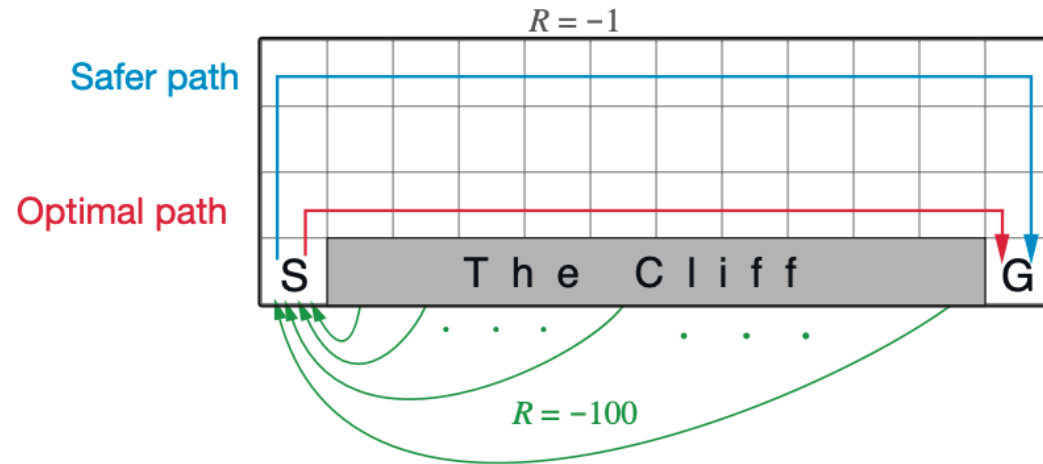
$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Pseudo-code

Example – Cliff walking



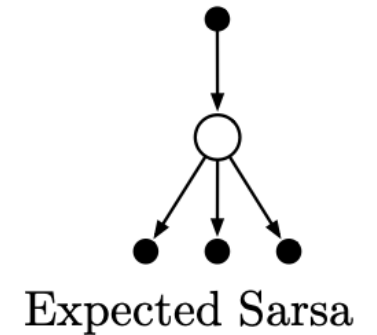
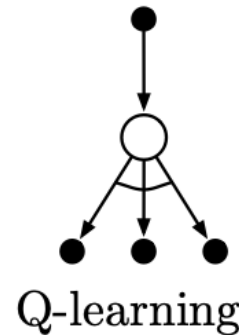
- Undiscounted
- Episodic tasks
- Reward of -1 in white; -100 in the cliff (gray)
- Both Sarsa and Q-learning with epsilon of 0.1

Expected Sarsa

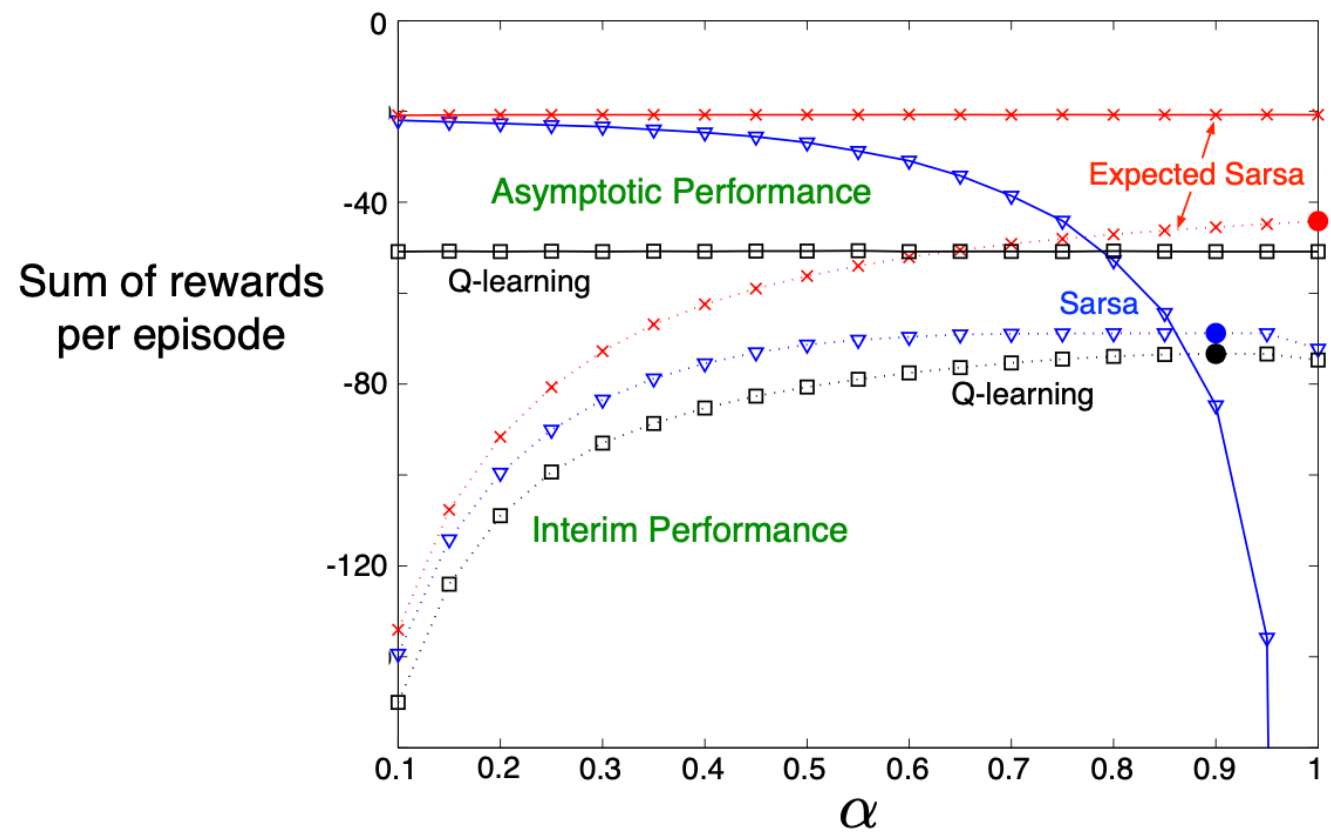
- Similar to Q-learning but instead of the maximum over next state action-pairs, it uses the expected value under the current policy

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \mathbb{E}_{\pi} [Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t) \right] \\ &\leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right], \end{aligned}$$

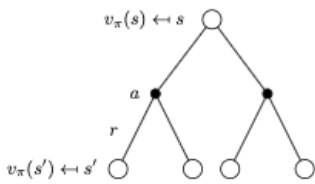
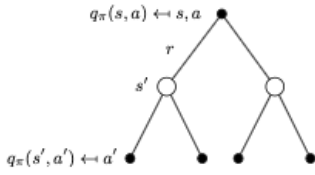
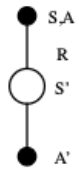
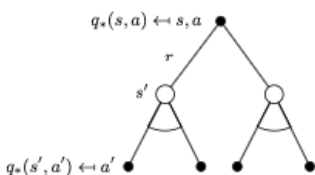
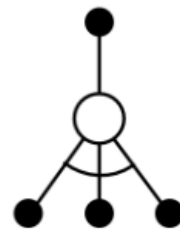
- Computationally, more intensive
- Eliminates variance



Example – Cliff walking



Relationship of DP with TD

	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Bellman Expectation Equation for $v_{\pi}(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_{\pi}(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_{*}(s, a)$	 Q-Value Iteration	 Q-Learning

Relationship of DP with TD

<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Iterative Policy Evaluation $V(s) \leftarrow \mathbb{E}[R + \gamma V(S') \mid s]$	TD Learning $V(S) \stackrel{\alpha}{\leftarrow} R + \gamma V(S')$
Q-Policy Iteration $Q(s, a) \leftarrow \mathbb{E}[R + \gamma Q(S', A') \mid s, a]$	Sarsa $Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma Q(S', A')$
Q-Value Iteration $Q(s, a) \leftarrow \mathbb{E}\left[R + \gamma \max_{a' \in \mathcal{A}} Q(S', a') \mid s, a\right]$	Q-Learning $Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma \max_{a' \in \mathcal{A}} Q(S', a')$

where $x \stackrel{\alpha}{\leftarrow} y \equiv x \leftarrow x + \alpha(y - x)$

Summary

